



Redundancy

In this lesson, we will explore redundancy as a high availability mechanism.

We'll cover the following

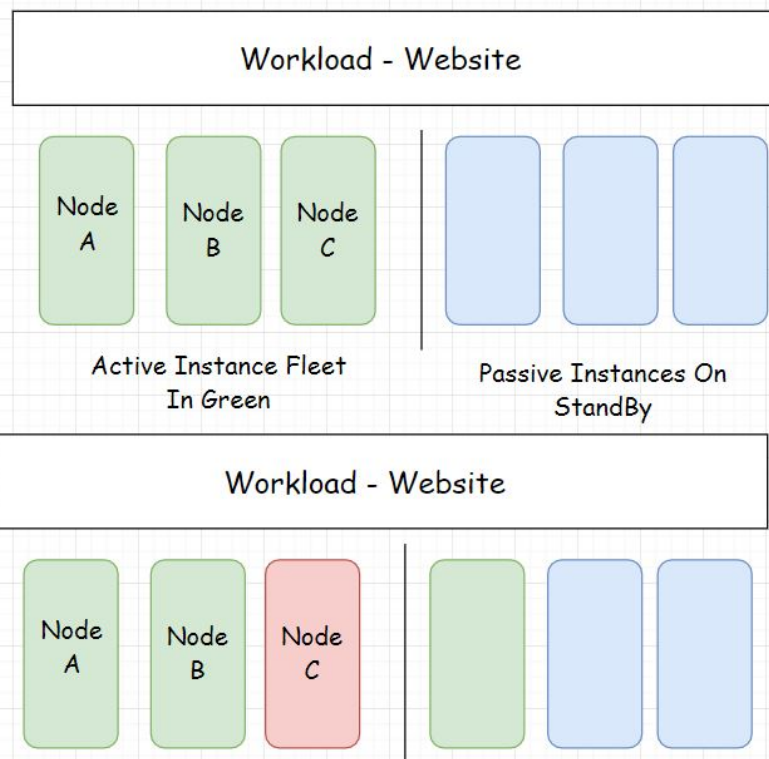


- Redundancy – Active-passive HA mode
- Getting rid of single points of failure
- Monitoring and automation

Redundancy – Active-passive HA mode

Redundancy is duplicating the server instances and keeping them on standby to take over in case any of the active server instances go down. It is the fail-safe backup mechanism in the deployment infrastructure.





The illustration above shows the redundant instances. One of the redundant instances takes over when the active instance goes offline.

This instance setup approach is also known as the *Active-passive HA* mode. An initial set of nodes are active, and a set of redundant nodes are passive, on standby. Active nodes get replaced by passive nodes in case of failures.

There are systems like GPS, aircraft, communication satellites, etc., that have zero downtime. The availability of these systems is ensured by making the components redundant.

Getting rid of single points of failure



Distributed systems became mainstream with large-scale applications solely because, in them, we can eliminate the single points of failure that were a big downside of a *monolithic architecture*. I discussed this in the previous lesson how *microservices* facilitate a *fault-tolerant* architecture.



In a *highly available* system, a large number of distributed server nodes work in conjunction with each other to achieve a single synchronous application state.



When so many redundant nodes are deployed, there are no single points of failure in the system. In case a node goes down, redundant nodes take its place. The system as a whole remains unimpacted.

Single points of failure at the application component level mean bottlenecks. I discussed this earlier, having just one database instance handling requests from a number of application nodes. We should detect bottlenecks in performance testing and get rid of them as soon as possible.

Monitoring and automation

Systems should be well monitored in real-time to detect any bottlenecks or single point of failures. Automation enables the instances to self-recover without any human intervention. It gives the instances the power of self-healing.

Also, the systems become intelligent enough to add or remove instances on the fly as per the requirements. *Kubernetes* is one good example of this.

Since the most common cause of failures is human error, automation helps

